



Final Reflection

FOOL'S DESCENT

Miguel Eduardo Vega Bisonó - A01781892

Regina Fernanda Portela Palacios - A01786698

Santiago Aguilar Mello - A01754105



STRATEGY

To make this game we followed the following steps:

1. First we each gave an idea during the brainstorm. Fernanda said we should implement tarot cards. Santiago had the idea of using sun and moon cards as empty shells/bullets like in russian roulette. Miguel had the idea of a randomly generated map like in Inscryption.
2. Using the combination of these ideas the next step was talking over as a team what artstyle and vibe should the game have.
3. Then once we had everything the game would have aswell as how the duel would work we started implementing it.
4. We tried to divide the work as equally as possible each member doing approximately $\frac{1}{3}$ of the complete game, each member working on database, backend and frontend equally so that everyone had a solid grasp on what was happening in the game
5. Using SCRUM we implemented mechnics and fixed bugs using issues as a checklist which made programming really intuitive.
6. With this strategy, we were able to completely implement everything we wanted in the game in an efficient way.

CHALLENGES WE FACED

While coding and implementing mechanics we faced some challenges that put our strategy to the test.

Our first challenge was during the programming of the duel, we originally only had one .js file with the complete game logic which had grown to become more than 2000 lines long. The problem was that the debugging process was very complicated because while trying to find the line of code you wanted to change you would have to look at more than a thousand lines of code to find the one that needed changes. To solve this we divided the game into more than 20 files, one main.js file for each difficulty and one .js for each part of the code like game cards and player turns. All of this while complicated at first really helped to implement future mechanics much more easily since every part was neatly classified.

Our other challenge was the connection of the database with the actual game flow. At first, most of the game information existed only inside JavaScript variables, like the player coins, cards, map position, upgrades, and duel results. This worked while testing the game locally, but it became a problem once we needed the information to persist after changing pages or closing the game. To solve this, we created a backend using Express and connected it to a MySQL database. The frontend sends information through API endpoints, and the backend saves or loads the correct data using SQL tables and stored procedures. This was challenging because we had to understand how the frontend, backend, and database communicate with each other, but it allowed us to save player progress, retrieve previous runs, track statistics, and connect upgrades and cards to each player account.

LESSONS LEARNED

Some lessons learned from this were that planning the structure of the project early is very important, because once the game grows, having all the logic in one file makes debugging and teamwork much harder. Dividing the code into smaller files helped us understand where each mechanic belonged and made future changes less confusing.

We also learned that a database is not just a place to store information, but a system that has to communicate correctly with the rest of the game. The frontend, backend, API endpoints, and SQL database all depend on each other, so a small error in one part can affect the whole gameplay flow. Because of this, we learned the importance of testing each connection step by step, keeping data organized, and making sure the game state can be saved and restored consistently.



PERSONAL CONCLUSIONS

Santiago: Through this project, I strengthened my understanding of how a videogame can connect with a database and backend system. I helped transform the game from a local experience into one that can save player information, progress, statistics, cards and upgrades. This showed the importance of organizing data correctly and creating reliable communication between the frontend, backend and database.

Fernanda: With this project, I developed a stronger understanding of interface design and user experience in a game environment. I helped make the game easier to navigate, from menus and tutorial-style screens to map interactions and visual feedback. This showed the importance of making mechanics clear to the player, not only functional in the code.

Miguel: I deepened my understanding of gameplay programming, especially in the duel system. Implementing card effects, enemy turns, animations, timing, and edge cases showed how complex game logic can become even in a simple-looking card game. This project helped reinforce the importance of modular code, debugging, and designing mechanics that feel fair and readable to the player.

